Assignment 3

In this assignment, you will learn how to load objects from disk, render instances and some basic scene composition to make a nice animation.

IMPORTANT NOTES:

- The solution to this assignment has been tested on a computer with integrated GPU (meaning very low spec GPU) and it runs well. Therefore, if you experience performance issues, it is NOT the hardware which is the problem.
- You are NOT allowed to change the source code provided in `obj.h` and `obj.c` files.
- The code should be able to run indefinitely without running out of memory on the GPU or the computer. No overflow exceptions should occur either.

Part 1 - 3D Object Loading and Instancing

In the first part of this assignment, we aim to load 3D objects from a file (including materials and textures) and render them correctly on the screen. There are many different file formats for storing 3D meshes and their material properties. In this assignment we will be using the **OBJ** file types and their accompanying **MTL** and texture files.

We will *not* spend time writing our own code for loading meshes and materials from these files ourselves. Instead, you are provided with two files: `obj.h` and `obj.c` developed by Robert Kooima. The files are slightly modified from the original version[1] to make them easier to use with the course's lab skeleton.

Understand the code

Download the aforementioned files from Canvas and add them to your current project. This library represents a mesh in a structure called struct `obj`, try to locate it in the files and look at the contents. Parts of it will seem familiar to you, there are also comments to help to understand the data structures used in the library.

You should also familiarise yourself with the material in objlib_quickstart.pdf.

---

[1] Available at https://github.com/rlk/obj

Load and render objects to the screen

The aim is to stop using the Mesh struct that have in our current code and use the struct obj instead, as that data type offers more properties for our meshes.

Load the object

For this part we have provided you with a zip file called `Cadillac.zip` which contains a 3D mesh of a car, its materials and textures. We will use this file for the rest of the assignment.

First comment out all the `insertModel` function calls. This way we know that none of the meshes that we used to define before are added to the scene.

Now is the time to load an object using the new library. You might find the following functions useful:

- `obj_create`: given a path to an `obj` file, it loads the mesh and its related materials and textures and returns a pointer to the object. If the pointer is null, then the load has not been successful.
- `obj_proc`: after a successful load of an object, we need to call this function so that the library does all the necessary preprocessing on the object.

Add texture support to object shaders

Now that we have a more complex object to render, we must redefine our shaders as well. Specifically, our shaders now must be able to handle textures as well. This means that an object might have a texture that defines its diffuse, specular, or even its normal values. Note that part of an object might have a texture, and other parts might only have a simple colour as before, therefore it is our job to make sure that the shader understands if it should use the texture colour or the diffuse colour values.

When the shaders are expanded to include the textures, we can move on to binding the shader variables to their respective values in the struct `obj`. The following functions will be useful:

- `obj_set_vert_loc`: The `obj` library handles the creation of VAO, VBO and IBO objects for us, but it does not know to what shader variable names it should bind them. The `obj_set_vert_loc` function allows us to provide this information to the library. Read about it in the README (or objlib_quickstart.pdf) file.
- `obj_set_prop_location`: Similarily, the `uniform` shader variables, representing different material properties of the object, also need to be

introduced to the library. Read about this function as well in the README (or objlib_quickstart.pdf) file.

Rendering to screen

By now we have (1) loaded the object, (2) pre-processed the object, (3) written the new shader code, and (4) linked the shader variables to `obj` values. The final remaining task is to send the `obj` to the pipeline and see the results on the screen.

This can be achieved by calling the `obj_render` function. If you have done everything correctly, and given a camera location of (0, 0, 5), camera orientation of (0, 0, 0), light location of (-20, 10, 10), and light colour of {amb: (0.1, 0.1, 0.1), diff: (1,1,1), spec: (1,1,1)} you should see a render as presented in Figure 1. Note: The assignment figures are rendered with shininess constant = 64 for all materials!

For a more appealing result. Let's move the car to (-1,-1,0) and rotate it by (-90,215,0) to get a better view of the car.

Instantiation

The object that we just loaded and rendered looks cool and all, but what if we want to draw many instances of it on the screen, each at a different location, with a different orientation, scale, and animations? Doing so would require us to think about several issues such as:

- Should we load the object several times? (think of load times, the redundant data stored in VAO, VBO and IBO, all the textures that are loaded many times, etc).
- Where do we store the local transformation vectors/matrices? (Should they be stored in the `obj` structure? What if we did not have the source code for the object loader, but instead had it as a single compiled library?)

Try to answer the above questions for yourself and come up with a well-designed data structure that allows you to create many instances of an object. Each instance can have its own transformations and be rendered with a different shader.

Then implement that data structure and show that it works by creating 2000 instances of the object and render them in a cube shape using the following locations, scale all of them uniformly to 0.3:

• X: starting from -20 to 18, steps of 2

• Y: starting from -20 to 18, steps of 2

• Z: starting from -5 to -13, steps of –2

Animation

You will need to adapt the animations so that each instance can be animated in different ways. To avoid accumulating floating point inaccuracies, and to allow rendering the animation in parallel, the animation shall now <u>depend only on</u> the given frame number.

To achieve this, each instance will need to track its original transformation vectors and the amount that each property (location, scale, rotation) should be animated per frame.

Using translation as an example, you will therefore need to write the necessary code that allows you to:

- Define an initial translation value for the object instance.
- Define a translation animation vector for the object instance.
- Write a function that, given a frame number, returns the correct translation matrix.

- You may need some property that allows the animation to reset after a certain number of frames, to account for animation looping.

Part 2 – Putting it together

We shall now take the knowledge learned throughout the assignments and apply it for rendering a nice looping scene.

For this part you are provided with 3 extra models: a lamp post, a road section and a skybox. Given the models that we have provided, implement an animation of a car driving down an endless road with terrain and repeating objects on both sides of the road. The scene shall be viewable from at least two viewpoints, including first person (e.g., what the driver sees) and third person. Objects on the side of the road shall include light sources, such as lamp posts. It shall be possible to turn the car's spotlights on and off (See: https://learnopengl.com/Lighting/Light-casters).

Some reference images are shown in the figures below as an example.

https://www.vecteezy.com/video/20397758-robot-driving-car-sci-fi-background

https://www.vecteezy.com/video/20389504-neon-glowing-hud-palms-futuristic-background

https://www.youtube.com/watch?v=ojVhM9poCZ4

First Person View

3rd Person View

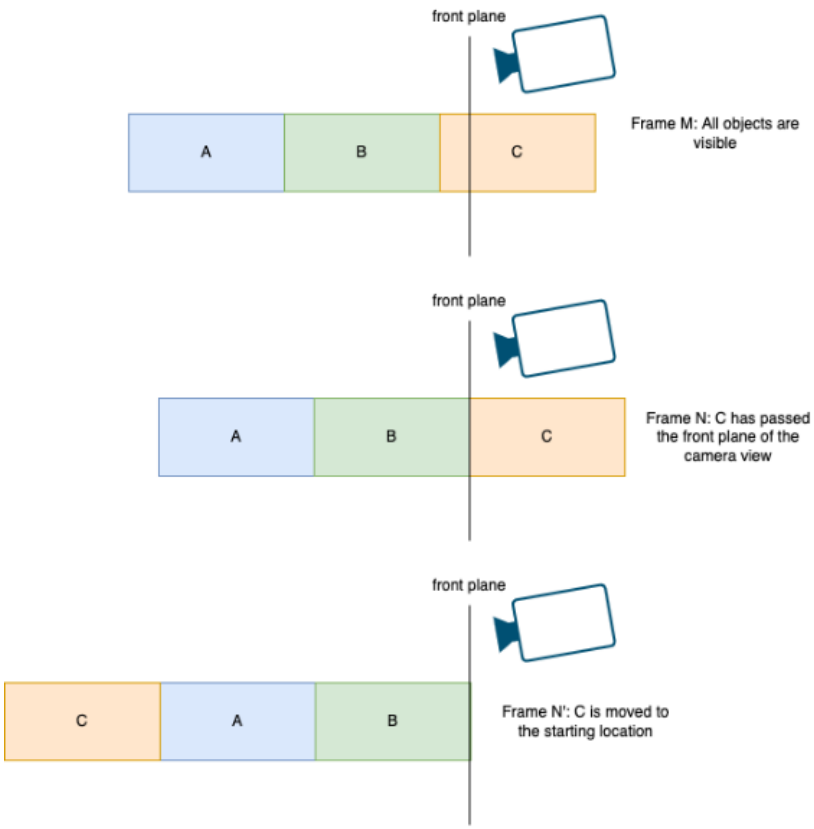
Scene Composition

During the scene composition process, recall the following: the camera is not actually moving. Instead, animate the world so that it moves past the camera.

You will have a scene with several light sources of different types. Your shaders need to account for this.

Infinite Scrolling

An infinite scrolling effect can be achieved by dividing the scrollable elements (e.g., the terrain) into tiles that are moved in the opposite direction of the scene's movement. When a tile completely passes the viewpoint of the camera, it is moved back to the starting location. See the Figure below for a graphical explanation of this process.



Scrollable objects in a scene.

Consider how to achieve this, given that the only input is the frame in the sequence that is being rendered. **Hint**: There are mathematical operations that map values into periodical ranges such as [-1,1] and [0, n).

Extras

Geometry Shader: Write a geometry shader that raises or lowers some points on the mesh during runtime. Use this technique to create dynamic and varied terrain that is different every time the program runs (2 EP). Hint: learn about Perlin Noise from chapter 11 in the book.

Cube Map: Now that we have a skybox we can employ environment mapping to reflect the skybox on the car within the scene (1 EP).

For the ambitious student: animate the skybox by generating the texture during runtime (1 EP).

Lighting Effects: Implement advanced lighting effects bloom and blur to enhance the visual quality of the scene. Bloom can add a glow effect to bright areas, making the scene more vibrant (2 EP).

Shadows: Use shadow mapping to render the shadows in the scene. For this part only the 3 closest lamp posts will create shadows (3 EP).